



Tower Siege
Relazione per il progetto di
“Programmazione ad
Oggetti”

A.A. 2025/26

Candidati:
Leonardo Rossi
Elia Naldini
Michel Gad
Stefano Fedele

Indice

1	Analisi	3
1.1	Descrizione e requisiti	3
1.1.1	Requisiti funzionali	3
1.1.2	Requisiti non funzionali	4
1.2	Modello del dominio	5
2	Design	6
2.1	Design dettagliato	8
2.2	Linee guida comuni	8
2.2.1	Leonardo Rossi— Ondate, movimento nemici, HP e danni	9
2.2.2	Elia Naldini-Torri: posizionamento, attacco, upgrade, interazione coi nemici	13
2.2.3	Michel Gad-Mappe, percorsi, multi-mappa, schermata iniziale	16
2.2.4	Stefano Fedele— Risorse, base HP, vittoria/sconfitta, punteggio, salvataggi	20
2.3	Riassunto scelte architetture e pattern software usati.	25
3	Sviluppo	26
3.1	Testing automatizzati	29
3.2	Package model	29
3.3	Package controller	31
3.4	Note di sviluppo	32
4	Commenti finali	32
4.0.1	Leonardo Rossi	33
4.0.2	Elia Naldini	34
4.0.3	Michel Gad	34
4.0.4	Stefano Fedele	35
4.1	Sviluppi futuri	36

4.1.1	Gameplay e contenuti	36
4.1.2	Meccaniche di gioco	36
5	Guida Utente	38
5.1	Avvio di una nuova partita	39
5.2	Schermata di gioco	40
5.3	Mappe di gioco	42
5.4	Tipologia di nemici	43
5.5	Tipologia di torri	44
5.6	Pulsante di controllo	45

1 Analisi

1.1 Descrizione e requisiti

Il software TowerSiege è un videogioco strategico di tipo Tower Defense il cui obiettivo principale consiste nel posizionare tatticamente diverse strutture difensive per respingere ondate periodiche di mostri e nemici. L'esperienza di gioco si articola su tre livelli a difficoltà crescente. All'inizio di ogni livello, il giocatore parte con un capitale iniziale di 250 monete e 100 punti vita. Le monete vengono utilizzate per acquistare fino a quattro tipologie di torri distinte, ciascuna caratterizzata da abilità uniche, statistiche specifiche e costi monetari proporzionali alla propria potenza, che possono essere edificate esclusivamente all'interno di appositi spazi di costruzione. Una volta definita la disposizione tattica iniziale, il giocatore può avviare l'attacco premendo il tasto W. Un'ondata si considera conclusa soltanto quando tutti i nemici presenti sul campo vengono eliminati, mentre ogni unità avversaria che riesce a raggiungere la fine del sentiero infligge 10 danni alla salute del giocatore. Ciascun livello è composto da un totale di cinque ondate e viene superato con successo se si riescono a sconfiggere tutti i nemici conservando i punti vita sopra lo zero. A supporto della difesa, il giocatore ha inoltre a disposizione due abilità speciali globali: la pioggia di fuoco, che infligge danni diretti a tutti i nemici presenti sulla mappa, e il gelo globale, che ne rallenta temporaneamente l'avanzamento.

1.1.1 Requisiti funzionali

- Gestione delle ondate di nemici con un movimento scorrevole
- Gestione delle torri, del loro posizionamento e dei loro attacchi
- Gestione della mappa e dei percorsi dei nemici
- Sistema di danno ai nemici

- Condizioni di vittoria e sconfitta
- Sistema di posizionamento delle torri
- Gestione dei punti vita (base, torri, nemici)
- Sistema di risorse (monete)

1.1.2 Requisiti non funzionali

- Corretto funzionamento in ciascuno dei 3 sistemi operativi principali: Linux, Windows e MacOS
- Interfaccia utente intuitiva per la gestione delle azioni di gioco
- Performance ottimizzate per garantire una migliore esperienza di gioco
- Architettura MVC per facilitare l'estensione del gioco con nuove funzionalita o la manutenzione

1.2 Modello del dominio

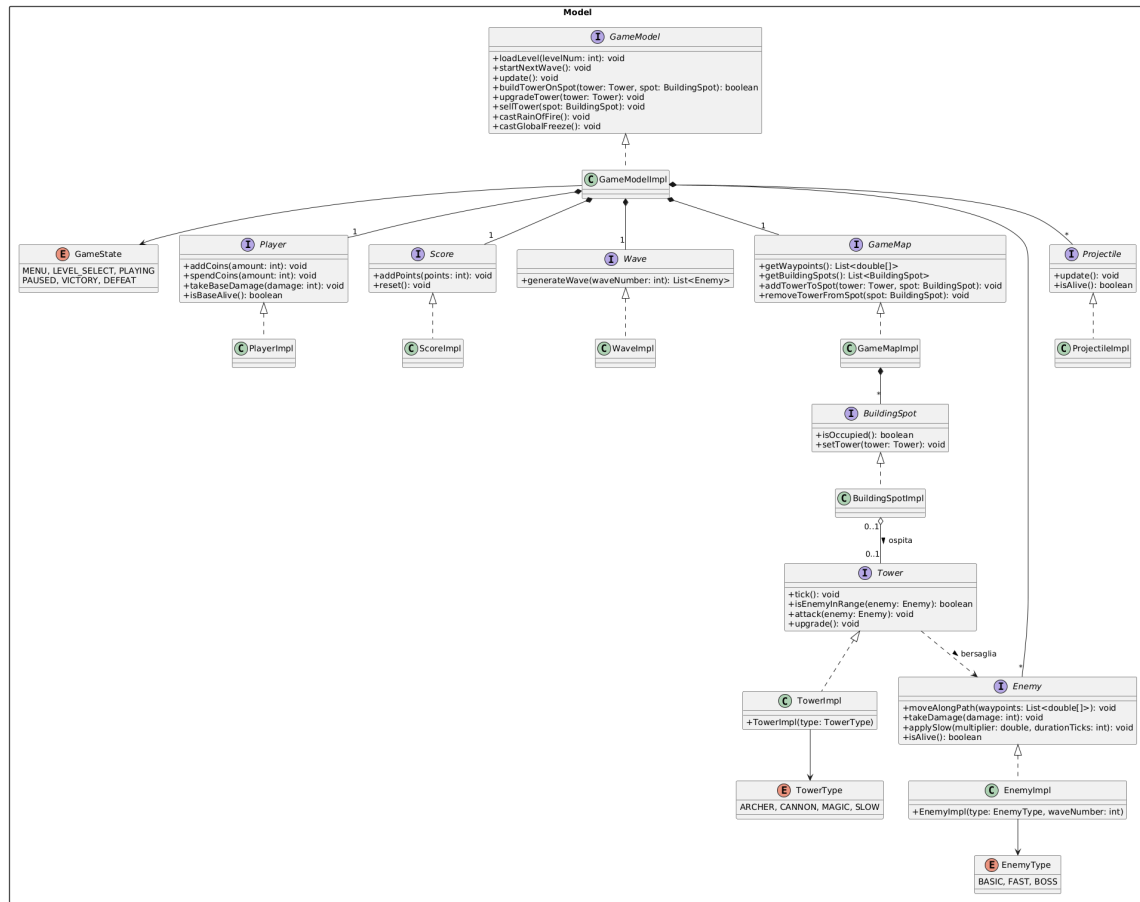


Figura 1: UML model

Il modulo Model costituisce la logica centrale di TowerSiege, gestendo lo stato del gioco, le regole di combattimento e l'economia in modo completamente disaccoppiato dalla grafica. Tramite l'entità principale GameModel, la classe coordina l'avanzamento dei livelli, l'attivazione delle abilità speciali e la gestione di punti vita e monete del Player. Si occupa inoltre di tracciare la mappa di gioco (GameMap) con le piazzole edificate (BuildingSpot), calcolando il posizionamento e gli attacchi delle torri (Tower). Infine, gestisce la generazione delle ondate (Wa-

ve) e il movimento dei nemici (Enemy) lungo il percorso, calcolandone danni e ricompense. Il tutto è strutturato attraverso interfacce ben definite per garantire massima modularità e manutenibilità del codice.

2 Design

Architettura Generale

L'applicazione segue il pattern architetturale **Model–View–Controller** (MVC) con una separazione netta delle responsabilità:

Model: Contiene l'intero stato del gioco e tutta la logica di business (gestione di nemici, torri, ondate, mappa, giocatore e punteggio). Ogni componente di questo livello è organizzato rigorosamente secondo il pattern Interface-Implementation (suddiviso nei package `api/` e `impl/`), favorendo il disaccoppiamento e la testabilità. Il punto cardine del Model è la sua interfaccia principale, che ha il compito di richiamare ed effettuare i controlli sui metodi delle singole classi, fornendo al resto del sistema un insieme coerente di azioni di gioco.

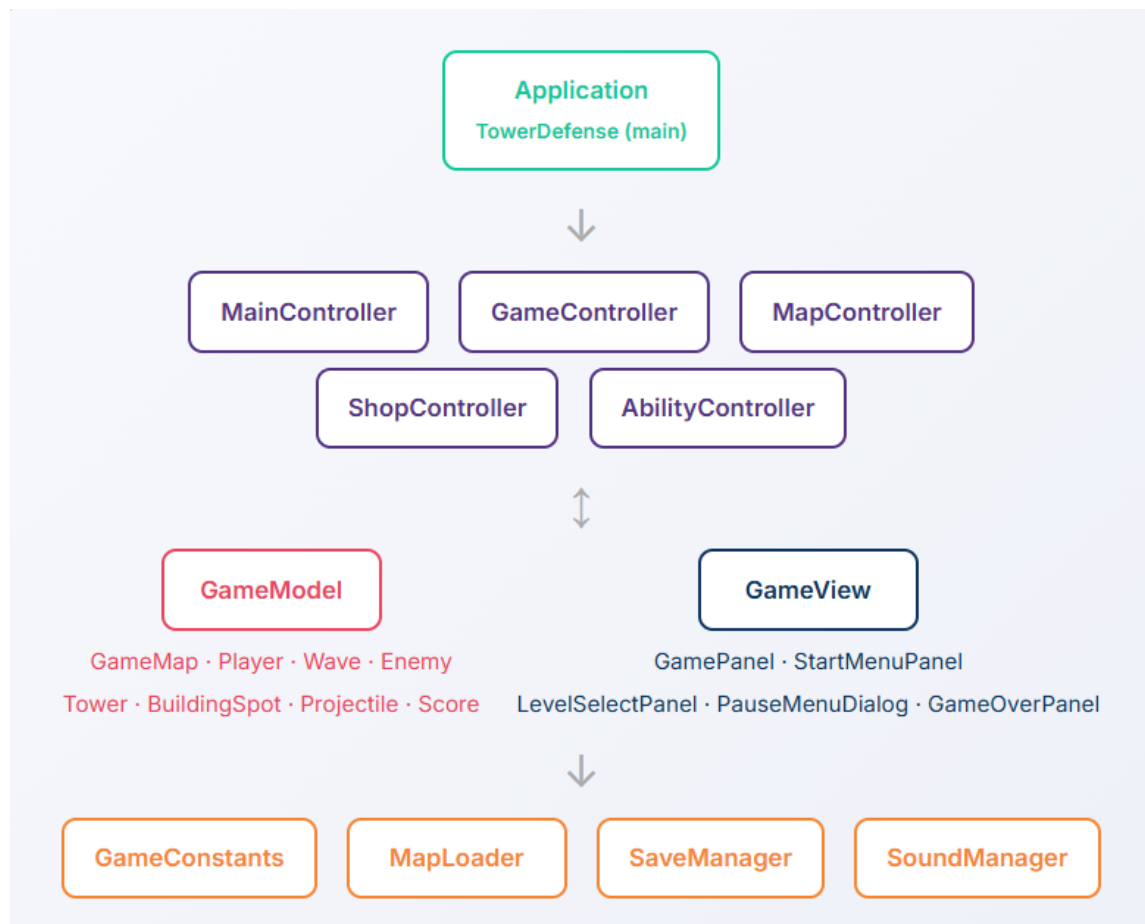
View: Si occupa del rendering grafico tramite librerie standard (Swing/Java2D), mantenendosi totalmente priva di logica di gioco. Il punto di entrata dell'interfaccia è il menu iniziale (es. `StartMenuView`), dal quale l'utente può interagire per avviare la partita. L'azione di avvio delega al `MainController` la creazione e la visualizzazione del vero elemento principale della UI: la `GameView`. Questa istanzia i pannelli e la componentistica grafica necessari a rendere visibile lo stato attuale del Model, permettendo l'interazione del giocatore tramite i controller corrispondenti.

Controller: Orchestra il flusso dell'applicazione, gestisce l'input dell'utente e coordina il game loop. Il componente principale di questa

sezione è l'interfaccia MainController, la quale rende possibile l'utilizzo di controller minori, ciascuno delegato alla gestione di uno specifico elemento della View. Il MainController ha inoltre il compito di inizializzare l'istanza del Model e di garantire il corretto avvio, scorrimento e salvataggio della partita.

A supporto dell'intera architettura, il package commons raccoglie tutte le classi di utilità trasversali (costanti, caricamento delle mappe e gestione dei salvataggi).

Tramite il corretto utilizzo del pattern MVC, si ottiene un sistema altamente flessibile: un'eventuale evoluzione o sostituzione totale della componente grafica (View) risulta agevolata e non richiede alcuna modifica alle sezioni di Model e Controller, non essendoci dipendenze dirette verso di essa.



2.1 Design dettagliato

2.2 Linee guida comuni

- **Pattern architetturali:** Utilizzo del pattern MVC, con rigorosa separazione tra `api` e `impl` per ogni sottodominio del `model`, e implementazione di controller dedicati per la gestione del gameplay e della UI.
- **Timestep logico:** Gestione discreta del tempo all'interno del `GameController`, con conseguente propagazione degli update al `GameModel`.

- **Disaccoppiamento View/Model:** Impiego di eventi o observer leggeri per garantire l'aggiornamento della View senza introdurre un accoppiamento stretto.
- **Gestione dei dati:** Centralizzazione dei dati condivisi all'interno del modulo `commons` (tra cui `gameconstants`, `mapdata` e `maploader`), con la logica dei salvataggi delegata a `commons.savemanager`.

2.2.1 Leonardo Rossi— Ondate, movimento nemici, HP e danni

Problema e Requisiti

Il sistema dei nemici costituisce il cuore del gameplay: i nemici vengono generati in ondate di difficoltà crescente, si muovono lungo un percorso definito da waypoint, possiedono punti vita scalati in base al numero dell'ondata, possono ricevere danni e rallentamenti, e danneggiano la base se raggiungono la fine del percorso.

Scelte Progettuali

Interfaccia Enemy Definisce il contratto per tutte le tipologie di nemico. Espone metodi per:

- **Ricevere danno (`takeDamage`):** riduce i punti vita e attiva un effetto visivo di "flash" per 8 tick.
- **Subire rallentamento (`applySlow`):** applica un moltiplicatore di velocità per una durata finita in tick.
- **Muoversi lungo il percorso (`moveAlongPath`):** segue i waypoint in coordinate pixel con interpolazione lineare, restituendo `true` quando il nemico ha raggiunto la fine.

- **Aggiornare lo stato** (`updateStatus`, `tickVisuals`): decrementa i timer di `slow` e `flash` ad ogni tick.

L'uso di un'interfaccia permette di aggiungere in futuro nuove tipologie di nemici (es. boss, nemici volanti) senza modificare il codice esistente (*Open/Closed Principle*).

Classe `EnemyImpl` Implementa `Enemy` con i seguenti campi di stato:

- `maxHealth` e `health`: calcolati nel costruttore come `type.getHealth() + (waveNumber * 8)`, garantendo la scalabilità progressiva della difficoltà.
- `baseSpeed` e `slowMultiplier/slowDurationTicks`: la velocità effettiva è calcolata come `baseSpeed * slowMultiplier`, separando la velocità intrinseca dal modificatore temporaneo.
- `currentWaypointIndex`: indice del prossimo waypoint da raggiungere. Il movimento è fluido grazie all'interpolazione `pixelX += dx/dist * step`.

Enum `EnemyType` Raccoglie le statistiche base dei tre tipi di nemico in modo immutabile, evitando magic number dispersi nel codice:

Tipo	Salute	Velocità	Ricompensa	Descrizione
BASIC	80	1	10 monete	Orco Raider
FAST	45	2	15 monete	Goblin Veloce
TANK	200	1	25 monete	Orco Bruto

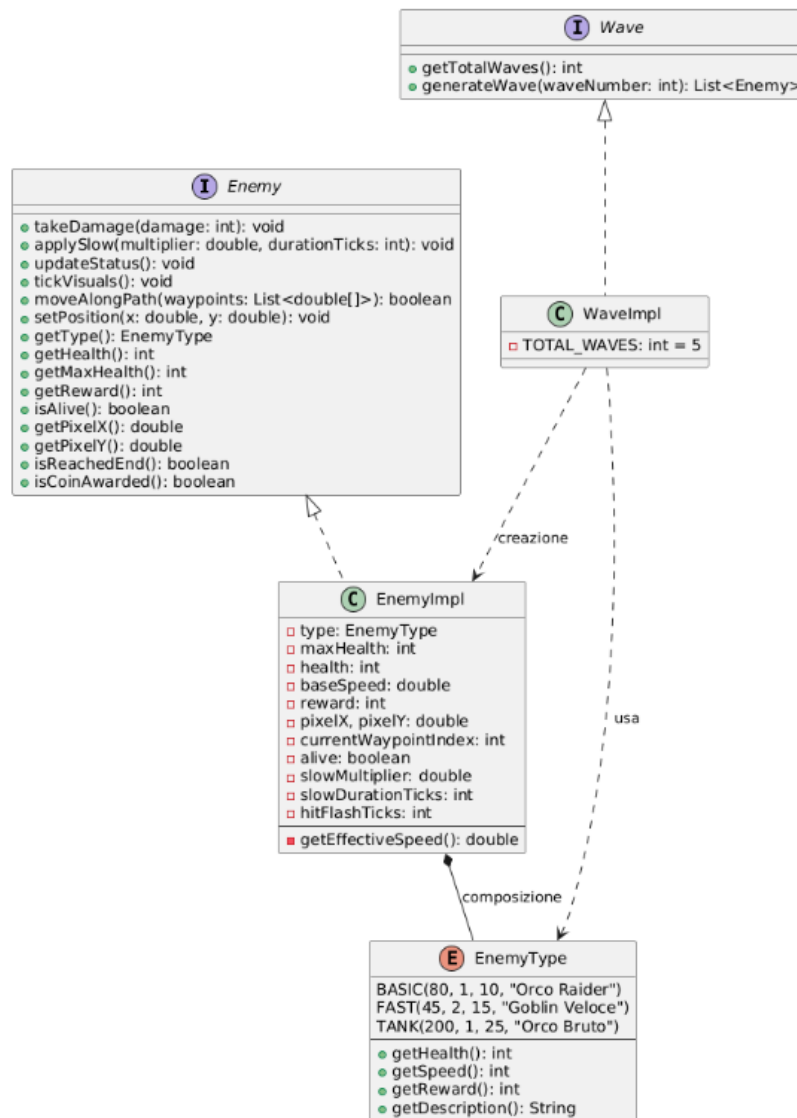
Tabella 1: Statistiche base definite nell'enum `EnemyType`

Interfaccia Wave e classe WaveImpl Generano le sequenze di nemici per ciascuna ondata. La composizione è determinata alitmicamente:

- Le ondate successive contengono più nemici: `numEnemies = 4 + waveNumber * 3`.
- I tipi **FAST** compaiono dall'onda 2, i **TANK** dall'onda 3.
- **WaveImpl** restituisce una `List<Enemy>` già istanziata, che il **GameModel** inserisce nella coda di spawn.

Interazione con il GameModel Nel metodo `update()` del **GameModel**:

- I nemici dalla `spawnQueue` vengono estratti uno al secondo (60 tick di delay) e posizionati al primo waypoint.
- Ad ogni tick, ciascun nemico vivo viene mosso lungo il percorso. Se raggiunge la fine, infligge danno alla base.
- I nemici uccisi (non per raggiungimento base) generano una ricompensa in monete e punti punteggio.
- I nemici morti vengono rimossi dalla lista attiva.



2.2.2 Elia Naldini-Torri: posizionamento, attacco, upgrade, interazione coi nemici

Problema e Requisiti

Le torri rappresentano lo strumento principale del giocatore per difendere la base. Devono poter essere piazzate su slot predefiniti, attaccare automaticamente i nemici nel proprio raggio d'azione, essere potenziate fino al livello 3, e generare proiettili che inseguono il bersaglio.

Scelte Progettuali

Interfaccia Tower Definisce il comportamento di una torre difensiva:

- `tick()`: avanza il cooldown interno di un tick. Ogni torre ha un proprio timer indipendente (gestisce il timer di attacco di una torre).
- `isEnemyInRange(Enemy)`: calcola se un nemico è entro la gittata usando la distanza euclidea in pixel. La gittata è convertita da unità griglia a pixel:

$$\text{rangePx} = \text{type.getRange()} \times 40.0$$

- `attack(Enemy)`: se il cooldown è a zero e il nemico è in range, crea e restituisce un `ProjectileImpl`. Altrimenti restituisce `null`.
- `upgrade()`: incrementa il livello della torre.
- `getDamage()`: il danno effettivo cresce linearmente col livello:

$$\text{type.getDamage()} + (\text{level} - 1) \times 2$$

Classe TowerImpl Implementa **Tower** con un design *stateless* rispetto al bersaglio: ovvero la torre non "ricorda" chi stava attaccando, ma valuta ad ogni tick tutti i nemici attivi. Questo semplifica la gestione della perdita del target (nemico morto o uscito dal range).

Enum TowerType Definisce 4 tipi di torre con statistiche immutabili:

Tipo	Costo	Danno	Gittata	Cooldown	Descrizione
BASIC	50	5	3	60 tick	Torre Cristallo
SNIPER	120	15	5	120 tick	Cavaliere
RAPID	80	2	3	20 tick	Mago
ICE	90	3	3	60 tick	Torre Ghiaccio

Tabella 2: Statistiche base definite nell'enum **TowerType**

Interfaccia BuildingSpot Rappresenta una cella della griglia su cui è possibile costruire. Espone coordinate griglia (`getCol`, `getRow`) e pixel centrali (`getPixelCenterX`, `getPixelCenterY`), stato di occupazione (`isOccupied`) e torre corrente (`getTower/setTower`).

Quando una torre viene piazzata (`setTower`), il `BuildingSpotImpl` invoca automaticamente `tower.setPosition(getPixelCenterX(), getPixelCenterY())`, centralizzando il calcolo della posizione.

Interfaccia Projectile e classe ProjectileImpl Un proiettile si muove a velocità costante (`SPEED = 15.0 pixel/tick`) verso il bersaglio. Al raggiungimento applica il danno della torre sorgente. Se la torre è di tipo `ICE`, applica anche il rallentamento (`SLOW_FACTOR = 0.5` per 20 tick). Il proiettile ha un offset di 20.0 pixel per colpire il centro visivo del nemico.



Ciclo di Attacco (in `GameModel.update()`) Il meccanismo di attacco si sviluppa all'interno del ciclo principale di aggiornamento secondo i seguenti passaggi:

1. Ogni torre riceve il metodo `tick()` per decrementare il proprio cooldown interno.
2. Per ogni nemico attivo, la torre verifica la propria portata tramite `isEnemyInRange()`.
3. Se il nemico è in range e il cooldown è a zero, il metodo `attack()` genera un `Projectile` che viene aggiunto alla lista attiva del model.
4. Ad ogni tick, `Projectile.update()` muove il proiettile verso il target. Al contatto, applica il danno ed l'eventuale effetto di rallentamento (*slow*).

Controller Associati La gestione dell'interazione utente è delegata ai seguenti controller:

- **MapController / MapControllerImpl:** Gestisce l'interazione del giocatore con gli slot della mappa:
 - **Click sinistro su slot vuoto:** crea una `TowerImpl` del tipo selezionato nello shop e invoca `model.buildTowerOnSpot()`.
 - **Click sinistro su torre esistente:** invoca `model.upgradeTower()`.
 - **Click destro:** invoca `model.sellTower()`.
- **ShopController / ShopControllerImpl:** Mantiene il `TowerType` attualmente selezionato dal giocatore all'interno del pannello negozio. Funziona come stato condiviso tra il controller della mappa e la vista.

2.2.3 Michel Gad-Mappe, percorsi, multi-mappa, schermata iniziale

Problema e Requisiti

Il sistema della mappa deve supportare una griglia 16×12 , definire il percorso dei nemici tramite waypoint, gestire gli slot di costruzione per le torri, e permettere il caricamento di mappe multiple da file JSON. Inoltre, la schermata iniziale deve fornire un'esperienza di navigazione coerente con menu, selezione livelli e regolamento.

Scelte Progettuali

Interfaccia GameMap Espone l'intero stato della mappa di gioco:

- `getGrid()`: restituisce una copia difensiva della matrice `int [12] [16]` (0 = erba, 1 = sentiero, 2 = costruibile), proteggendo l'incapsulamento.
- `getWaypoints()`: lista ordinata di coordinate pixel che definiscono il percorso dei nemici.
- `getBuildingSpots()`: lista di `BuildingSpot` su cui piazzare torri.
- `getDecorations()`: posizioni e tipi di decorazioni procedurali.
- `addTowerToSpot()` / `removeTowerFromSpot()`: gestiscono il piazzamento/rimozione delle torri sugli slot.

Classe GameMapImpl Il costruttore riceve le coordinate in formato griglia (da JSON) e le converte in coordinate pixel al centro della cella:

$$(\text{col} \times \text{TILE_SIZE}) + \left(\frac{\text{TILE_SIZE}}{2.0} \right)$$

Questo approccio separa la rappresentazione dei dati (griglia) dalla rappresentazione grafica (pixel), semplificando il rendering.

La griglia viene popolata marcando le celle attraversate dai waypoint come 1 (sentiero) e quelle con slot di costruzione come 2 (costruibile).

Classe MapData Classe contenitore utilizzata per la lettura e la conversione dei dati tra il formato JSON e gli oggetti Java della mappa. Contiene esclusivamente gli attributi e i relativi metodi, getter, setter per gestire parametri quali width, height, background, waypoints, buildingSpots e decorations, consentendo di separare la struttura del file di salvataggio dalla logica interna di gioco.

Classe MapLoader Carica e parse i file JSON delle mappe, sia da classpath (`loadFromClasspath`) che da filesystem (`loadMap`). Il parsing è implementato con espressioni regolari (regex), senza dipendenze da librerie JSON esterne, per:

- Estrarre campi interi (`extractInt`).
- Estrarre stringhe (`extractStringField`).
- Estrarre array 2D bilanciati (`extractDoubleArray2D` con `extractBalancedArray2D`).

Il supporto a mappe multiple è realizzato tramite la convenzione `level{N}.json` nella cartella `resources/maps/`. Il `GameModel` richiama:

```
MapLoader.loadFromClasspath("maps/level" + levelNum +  
                             ".json")
```

con fallback a `maps/map.json`.

Classe StartMenuPanel Panel Swing con sfondo a gradiente, titolo in oro, istruzioni e tre pulsanti (**INIZIA PARTITA**, **REGOLAMENTO**, **ESCI**). Il gradiente è realizzato sovrascrivendo `paintComponent` con `GradientPaint`.

Classe LevelSelectPanel Mostra i 3 livelli come nodi interattivi, distinguendo visualmente livelli sbloccati e bloccati in base al `maxUnlockedLevel` del model.



Formato JSON della Mappa Le mappe di gioco sono memorizzate in formato JSON all'interno della cartella **resources/maps/**. La struttura del file viene letta dal **MapLoader** e deserializzata nell'oggetto **MapData**.

I campi previsti nel formato JSON sono i seguenti:

- **width** e **height**: dimensioni della griglia di gioco in numero di celle (es. 16×12).
- **background**: identificativo dell'immagine o dello stile di sfondo della mappa.
- **waypoints**: array di coppie di coordinate $[x, y]$ in griglia che definiscono il percorso dei nemici.
- **buildingSpots**: array di coordinate $[x, y]$ dove il giocatore può costruire le torri.

- **decorations**: array di oggetti contenenti la posizione $[x, y]$ e il tipo di decorazione procedurale.

```
{
  "mapWidth": 800,
  "mapHeight": 600,
  "background": "",
  "waypoints": [[0, 5], [3, 5], [3, 2], ...],
  "buildingSpots": [[2, 4], [4, 6], ...],
  "decorations": [[1, 1, 0], [7, 9, 2], ...]
}
```

2.2.4 Stefano Fedele— Risorse, base HP, vittoria/sconfitta, punteggio, salvataggi

Problema e Requisiti

Il giocatore possiede risorse (monete) per acquistare e potenziare torri, una base con punti vita che i nemici possono ridurre, un sistema di punteggio che valorizza le performance, e la possibilità di salvare/caricare i progressi tra le sessioni.

Scelte Progettuali

Interfaccia Player Incapsula lo stato economico e difensivo del giocatore:

- **addCoins(int) / spendCoins(int)**: gestiscono il bilancio delle monete. **spendCoins** restituisce un booleano per verificare la disponibilità dei fondi, evitando saldi negativi.
- **takeBaseDamage(int)**: riduce i punti vita della base.

- `reset()`: ripristina monete e vita ai valori iniziali (da `GameConstants`).
- `isBaseAlive()`: condizione di sconfitta (`baseHealth > 0`).

Classe `PlayerImpl` Implementazione diretta che legge i valori iniziali da `GameConstants.STARTING_COINS` (250) e `GameConstants.BASE_HEALTH` (100). Il metodo `reset()` è dichiarato `final` per garantire che la semantica di `reset` sia consistente anche in eventuali sottoclassi.

Interfaccia `Score` e classe `ScoreImpl` Sistema di punteggio incrementale:

- I punti vengono assegnati alla morte di un nemico:

$$\text{enemy.getReward()} \times 10$$

- Alla vittoria, si aggiungono i bonus:

$$(\text{baseHealth} \times 10) + \text{coins}$$

- `addPoints` ignora valori negativi (validazione dell'input).

Condizioni di Vittoria e Sconfitta (in `GameModelImpl.update()`)

- **Sconfitta:** `player.getBaseHealth() ≤ 0` → stato `DEFEAT`.
- **Vittoria:** tutte le ondate completate, coda di spawn vuota e nessun nemico attivo → stato `VICTORY` con un delay di 50 tick prima del reindirizzamento alla selezione dei livelli.

Classe SaveManager Utility statica per la persistenza su file di testo (`save.txt`):

- `save(maxUnlockedLevel, bestScore)`: scrive su file con formato `key=value`.
- `loadMaxLevel()` / `loadBestScore()`: leggono i campi con un parser generico (`readField`).
- `saveExists()` / `deleteSave()`: gestiscono l'esistenza del file.
- Overload con percorso (*path*) personalizzato per facilitare il testing unitario.

Il salvataggio viene invocato alla vittoria in `GameModel`, dove:

```
SaveManager.save(currentLevel + 1, levelScore)
```

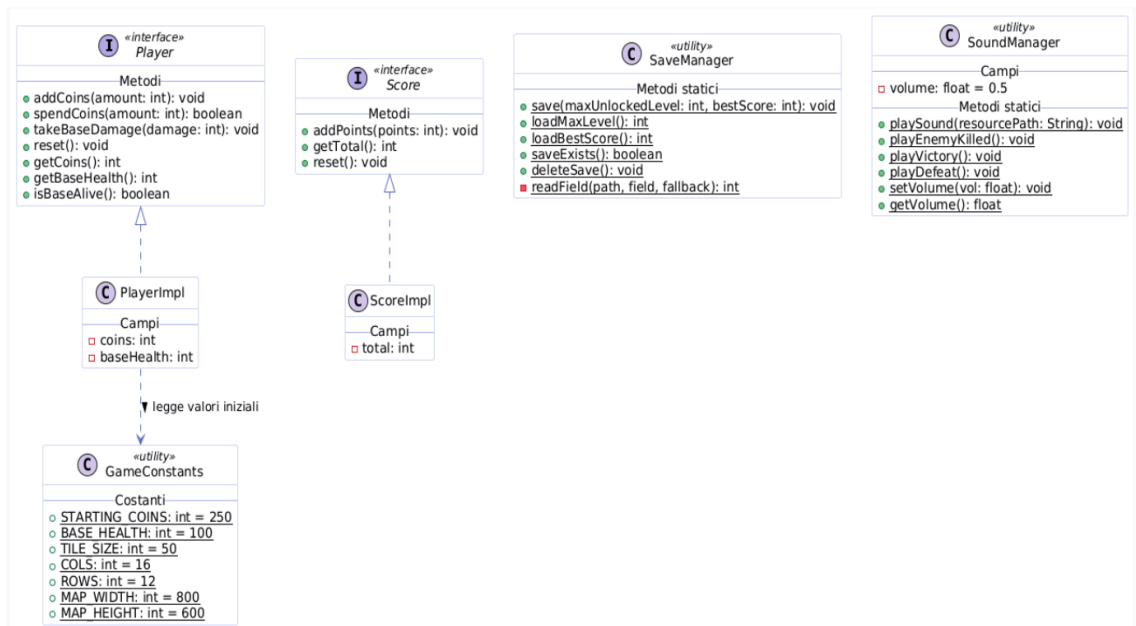
sblocca il livello successivo e registra il punteggio.

Classe SoundManager Utility statica per la riproduzione di effetti sonori tramite `javax.sound.sampled`: volume configurabile globalmente (0.0–1.0), tre effetti predefiniti (`playEnemyKilled()`, `playVictory()`, `playDefeat()`), e caricamento da classpath con gestione silenziosa degli errori.

Flusso Economico del Gioco

Il bilanciamento economico e la gestione dello stato di gioco seguono una sequenza ben definita:

1. **Inizio partita:** il giocatore parte con 250 monete e 100 HP alla base.
2. **Acquisto torre:** sottrae il costo (50–120 monete) dal bilancio.



3. **Nemico ucciso:** +10–25 monete, +100–250 punti punteggio.
4. **Potenziamento torre:** costo pari al 50% del prezzo base.
5. **Vendita torre:** rimborso del 50% del costo originale.
6. **Nemico raggiunge la base:** –10 HP alla base. Se $HP \leq 0 \rightarrow$ **SCONFITTA**.
7. **Tutte le ondate completate:** $\rightarrow$ **VITTORIA**. Bonus: $(HP \times 10) +$ monete residue.

GameModel come Orchestratore

La classe `GameModelImpl` è il punto centrale dove tutti i sotto-sistemi convergono. Nel suo metodo `update()`, eseguito 60 volte al secondo (60 FPS):

1. **Spawn:** estrae i nemici dalla coda con un delay di 60 tick ($\sim$ 1 secondo).
2. **Movimento:** aggiorna la posizione e lo stato di ogni nemico.
3. **Danno alla base:** se un nemico raggiunge la fine del percorso, infligge 10 HP di danno alla base.
4. **Proiettili:** aggiorna la traiettoria e rimuove i proiettili giunti a destinazione o inattivi.
5. **Attacco torri:** ogni torre verifica la portata e spara al nemico nel range se il cooldown lo consente.
6. **Ricompense:** assegna monete e punti al giocatore per ogni nemico eliminato.
7. **Pulizia:** rimuove i nemici morti dalla lista attiva.
8. **Condizioni finali:** controlla lo stato di sconfitta ($HP \leq 0$) o di vittoria (ondate completate, nessun nemico rimanente).
9. **Abilità speciali:** decrementa i cooldown delle abilità speciali (*Pioggia di Fuoco* e *Gelo Globale*).

2.3 Riassunto scelte architetturali e pattern software usati.

Pattern	Dove è applicato	Motivazione
MVC	Intera applicazione	Separazione delle responsabilità tra dati, presentazione e logica di controllo.
Interface–Implementation	Tutte le classi model e controller	Disaccoppiamento, testabilità, rispetto del <i>Dependency Inversion Principle</i> .
Enum con stato	EnemyType, TowerType, GameState	Raggruppamento <i>type-safe</i> delle configurazioni immutabili.
DTO (Data Transfer Object)	MapData	Trasferimento dati JSON → oggetti Java senza logica di business.
Utility Class	SaveManager, SoundManager, GameConstants	Funzionalità <i>stateless</i> condivise, costruttore privato.
Observer implicito (Timer)	GameControllerImpl	Il <code>javax.swing.Timer</code> funge da <i>game loop</i> che notifica model e view.
Façade	MainControllerImpl	Coordina tutti i sotto-controller e semplifica l'interfaccia verso la view.

Tabella 3: Pattern di design applicati all'interno dell'architettura del progetto

3 Sviluppo

Feature Avanzate Utilizzate

Rossi – Ondate e Nemici

Feature Avanzate Utilizzate

- **Uso di Enum con campi e costruttore parametrizzato (*Rich Enum*):** `EnemyType` è un enum che incapsula proprietà multiple (`health`, `speed`, `reward`, `description`) e metodi di accesso, andando oltre il semplice uso enumerativo `EnemyType.java` (R6-R58)
- **Lambda expressions:** Uso di lambda per `ActionListener` nel `GameOverPanel` e nel `PauseMenuDialog`, e per il game loop in `GameOverPanel.java` (R67) `GameControllerImpl.java` (R42)
- **Method reference (`::`):** Uso di method reference per il cablaggio (*wiring*) dei keybinding in `GameViewImpl`, passando `gc::togglePause`, `ac::forceNextWave`, `ac::triggerRainOfFire`, `ac::triggerGlobalFreeze` come `Runnable` `GameViewImpl.java`(R133–R139).
- **Uso di `javax.sound.sampled` (JDK Audio API):** Il `SoundManager` usa `AudioSystem`, `Clip`, `AudioInputStream` e `FloatControl.Type.MASTER_GAIN` per riprodurre effetti sonori con controllo del volume – libreria JDK non trattata a lezione `SoundManager.java`(R30–R52).

Gad – Mappa, Percorsi e Schermata Iniziale

Feature Avanzate Utilizzate

- **Uso di Stream con method reference:** Pipeline funzionale in `GameMapImpl.getTowers()` che combina `stream()`, `filter()`, `map()` e `collect()` con method reference `BuildingSpot::isOccupied` e `BuildingSpot::getTower`
`GameMapImpl.java` (R128-R133)
- **Uso di Stream con `Collectors.joining()`:** In `MapLoader.loadFromClasspath()` lettura di un file di risorse tramite `BufferedReader.lines().collect(Collectors.joining())` per convertire uno stream di righe in una singola stringa
`MapLoader.java` (R62)
- **Uso di `java.util.regex (Pattern/Matcher)` per parsing JSON manuale:** Il `MapLoader` implementa un parser JSON custom senza dipendenze esterne, usando `Pattern.compile()` e `Matcher` per estrarre campi interi, stringhe e array 2D dal JSON
`MapLoader.java` (L83-L131)
- **Uso di `java.nio.file (NIO.2)`:** Lettura di file con `Files.readAllBytes()` e charset esplicito `StandardCharsets.UTF_8`, parte della NIO.2 API
`MapLoader.java` (L44)
- **Lambda expressions:** Uso di lambda per gli `ActionListener` dei pulsanti nel menu iniziale e nel dialog delle regole.
`StartMenuPanel.java` (R101-R105)
`RulesDialog.java` (R61)
- **Uso di `javax.imageio.ImageIO` e `java.awt.Graphics2D` per rendering personalizzato:** Override di `paintComponent` con painting custom, gradiente, overlay semi-trasparente e caricamento immagini dal classpath.
`StartMenuPanel.java` (R140-R157)

Naldini – Torri, Proiettili e Potenziamenti

Feature Avanzate Utilizzate

- **Uso di Enum con campi e costruttore parametrizzato (*Rich Enum*):** TowerType incapsula cost, damage, range, cooldown e description con metodi di accesso.
TowerType.java (R6-R80)
- **Uso di java.nio.file (NIO.2) con try-with-resources:** In SaveManager, uso di Files.newBufferedWriter() e Files.newBufferedReader() con Paths.get() e StandardCharsets.UTF_8 per I/O su file in modo moderno e sicuro
SaveManager.java (R48-R132)
- **Uso di java.awt.image.BufferedImage per manipolazione pixel-level:** In ImageLoader.tintImg(), manipolazione diretta dei pixel ARGB per applicare un filtro di colore (*tinting*) alle immagini – tecnica avanzata di *image processing*
ImageLoader.java (L203–L232).

Fedele – Risorse, Punteggio e Salvataggio

Feature Avanzate Utilizzate

- **Lambda expression con removeIf (costrutti funzionali):**
In GameModelImpl.update(), uso di List.removeIf() con lambda per rimuovere proiettili e nemici morti in modo dichiarativo.)
GameModelImpl.java (R212–R215, R245).

3.1 Testing automatizzati

La suite di test di TowerSiege è stata realizzata con JUnit 5 e rispecchia fedelmente la struttura dei package di produzione: i test del Model si trovano sotto `src/test/java/it/unibo/towerdefense/model`, mentre quelli dei Controller sotto `src/test/java/it/unibo/towerdefense/controller`. L'esecuzione è completamente automatizzata tramite Gradle; il comando `./gradlew build test` avvia l'intera batteria senza interventi manuali, producendo i report di esito e copertura, con build conclusa con successo. Sul versante del Model abbiamo verificato le principali entità e regole di dominio del tower defense: la salute e le risorse del giocatore, la generazione e progressione delle ondate, il calcolo del percorso di mappa, l'applicazione dei danni da parte delle torri e l'avanzamento dei proiettili. Sul versante dei Controller abbiamo simulato l'interazione della UI: l'attivazione delle abilità globali (`AbilityControllerImplTest`), il flusso di acquisto dallo shop e le operazioni di posizionamento/potenziamento/vendita delle torri (`MapControllerImplTest`), nonché i casi di pausa/ripresa della partita quando rilevante. La copertura contempla sia i casi nominali (ad esempio, torre che spara a un nemico nel raggio) sia condizioni al contorno progettate per stressare la logica: spesa oltre le monete disponibili, nemici che escono dal range, cooldown delle abilità forzato, gestione di coordinate e collisioni in traiettorie diagonali. Questo impianto di test, insieme all'automazione di build, garantisce regressioni rapide, affidabilità delle funzionalità core e facilita l'estensione futura del gioco mantenendo la qualità del codice.

3.2 Package model

- **PlayerImplTest:** Verifica lo stato iniziale del giocatore, la corretta aggiunta e spesa delle monete (testando i limiti per impedire acquisti con fondi insufficienti), la gestione della perdita dei punti salute della base e l'efficacia del reset di fine partita.

- **ScoreImplTest:** Testa il funzionamento dell'accumulo del punteggio, assicurandosi che i valori negativi vengano scartati e verificando il corretto azzeramento del totale.
- **EnemyImplTest:** Verifica lo stato nominale del nemico, la ricezione dei danni, il movimento coordinato tra i waypoint spaziali (simulando l'avanzamento a tick) e la corretta applicazione, nonché successiva scadenza, degli effetti di rallentamento.
- **EnemyTypeTest:** Controllo strutturale per assicurare che le enumerazioni restituiscano i valori invarianti attesi per salute base, velocità e ricompensa in monete di ogni singola tipologia di nemico (Base, Veloce, Tank).
- **TowerImplTest:** Analizza il corretto calcolo dei danni incrementali dopo i potenziamenti, l'algoritmo di rilevamento dei nemici all'interno del raggio d'azione circolare e l'impossibilità di sparare prima della ricarica del cooldown.
- **TowerTypeTest:** Controllo strutturale analogo sui tipi di torre per verificarne il costo iniziale, il danno, la gittata e il rateo di fuoco nominali.
- **BuildingSpotImplTest:** Testa la piattaforma di interazione spaziale del campo di gioco, confermando il transito dallo stato "libero" a "occupato" e verificando la corrispondenza tra le coordinate logiche della matrice e quelle in pixel dello schermo.
- **GameMapImplTest:** Assicura la corretta interpretazione delle mappe generate (array 2D), il rilevamento dinamico degli spot di costruzione, la corretta immissione dei waypoint per i nemici e i controlli incrociati per impedire di posizionare torri su spot inesistenti o già occupati.

- **ProjectileImplTest:** Sottopone a verifica la fisica di inseguimento, simulando lo spostamento vettoriale del proiettile a ogni fotogramma verso un bersaglio mobile. Verifica inoltre la distruzione del proiettile in caso di scomparsa anticipata del bersaglio e la rigorosa applicazione del danno all’impatto.
- **WaveImplTest:** Controlla l’algoritmo di progressione della difficoltà, assicurando che la composizione del gruppo di nemici in termini di numerosità e tipologia cresca adeguatamente rispetto all’indice dell’ondata.
- **GameModelImplTest:** Esegue un collaudo dell’intero modello centrale. Verifica la stabilità delle transizioni tra i macro-stati del gioco (Menu, Partita in corso, Pausa), la funzionalità delle abilità AoE (Pioggia di Fuoco e Congelamento) e il rispetto dei tempi di ricarica.

3.3 Package controller

Nota di testing: I controller sono stati testati avvalendosi di classi "Dummy" (ovvero finti componenti grafici simulati, come `DummyGameView`), che permettono di testare le logiche interne di gestione senza invocare realmente le librerie grafiche di Java Swing.

- **ShopControllerImplTest:** Verifica la corretta registrazione da parte del sistema dell’entità attualmente selezionata per l’acquisto nel negozio, garantendo che selezioni invalide non corrompano lo stato dell’acquisto in corso.
- **MapControllerImplTest:** Testa il collegamento fra l’input utente e il sistema logico: verifica il successo o il fallimento (causa fondi insufficienti) della costruzione di una nuova torre, della sua

procedura di potenziamento (se cliccata nuovamente) e del suo smantellamento.

- **AbilityControllerImplTest:** Assicura che i comandi legati alle scorciatoie o ai pulsanti delle abilità (forzatura ondata, pioggia di fuoco, congelamento) vengano correttamente inoltrati per l'esecuzione al `GameModel`.
- **GameControllerImplTest:** Mette alla prova il controllo del loop di esecuzione a 60 FPS simulandone le sospensioni: verifica l'esatta inversione degli stati durante l'attivazione/disattivazione della Pausa e l'invio corretto del segnale di Reset dei timer in caso di riavvio del livello.
- **MainControllerImplTest:** Verifica l'orchestrazione primaria e la navigazione dell'utente all'interno dei menu. Si assicura che transizioni critiche, come il passaggio dalla schermata principale alla selezione dei livelli, e da quest'ultima al campo di battaglia, producano i cambiamenti di stato attesi dal sistema.

3.4 Note di sviluppo

4 Commenti finali

L'obiettivo principale di TowerSiege era costruire un tower defense completo e coerente, capace di coniugare strategia, gestione delle risorse e progressione su più livelli. La sfida è stata duplice: da un lato modellare in modo deterministico ondate, percorsi e interazioni torri-nemici; dall'altro mantenere il bilanciamento tra difficoltà crescente, varietà di torri e chiarezza dell'interfaccia. L'applicazione finale soddisfa i requisiti funzionali previsti: tre mappe con percorsi dedicati, cinque ondate per livello, quattro tipologie di torri con posizionamento su piazzole, attacco e upgrade, abilità globali (pioggia di fuoco e gelo),

economia a monete, punteggio e condizioni di vittoria/sconfitta legate agli HP della base. L'adozione rigorosa dell'architettura MVC, con interfacce stabili nel Model e controller specializzati per mappa, abilità e shop, ha permesso al team di lavorare in parallelo riducendo l'accoppiamento e rendendo la logica di gioco indipendente dalla UI. Questo ha reso il sistema più manutenibile, testabile (JUnit 5, esecuzione automatizzata via Gradle) ed estensibile: nuove torri o nemici, ulteriori livelli e perfino una diversa presentazione grafica possono essere introdotti intervenendo in modo localizzato, senza impatti trasversali sull'intero codice.

4.0.1 Leonardo Rossi

Il mio contributo si è concentrato sulla gestione delle ondate di nemici, sul movimento lungo il percorso e sul sistema di danni e punti vita. Per le ondate ho progettato la logica di spawn temporizzata, garantendo che ogni ondata rispettasse la configurazione del livello in termini di tipo e numero di nemici, con una struttura facilmente estendibile per introdurre nuove varianti. Per il movimento ho implementato un sistema di path-following a waypoint, in cui ogni nemico mantiene internamente l'indice del waypoint corrente e avanza in base alla propria velocità effettiva, tenendo conto di eventuali rallentamenti applicati dalle abilità globali tramite il metodo `applySlow`. Per i danni e i punti vita ho definito l'interfaccia `Enemy` con i metodi `takeDamage` e `isAlive`, gestendo il flash visivo all'impatto tramite `hitFlashTicks` e la notifica al `GameModel` alla morte del nemico per l'assegnazione della ricompensa in monete. Sono soddisfatto delle scelte adottate: separare la logica di avanzamento dalla logica di stato del nemico ha reso ogni componente testabile in isolamento e ha ridotto il rischio di effetti collaterali durante l'integrazione con il sistema di torri e abilità.

4.0.2 Elia Naldini

Il mio contributo ha riguardato il sottosistema torri nella sua interezza: posizionamento sulle piazzole, ciclo di attacco, potenziamento e interazione con i nemici. Per il posizionamento ho lavorato sull'interfaccia `BuildingSpot`, garantendo che ogni piazzola mantenesse il riferimento alla torre installata e che le operazioni di piazzamento e rimozione avvenissero in modo coerente con lo stato del `GameModel`. Per il ciclo di attacco ho implementato la gestione del cooldown interno alla torre, la selezione del bersaglio nel raggio d'azione e l'invocazione di `takeDamage` sul nemico selezionato, con supporto opzionale ai proiettili tramite l'interfaccia `Projectile`. Per il potenziamento ho definito una progressione a tre livelli con incremento delle statistiche e costo crescente, e una logica di vendita che restituisce un rimborso parziale proporzionato all'investimento. Ho coordinato `ShopController` e `MapController` per mediare tra le azioni della `View` e le operazioni sul `Model`, mantenendo la `View` priva di logica di dominio. Sono soddisfatto di queste scelte: l'uso di `TowerType` come configurazione centralizzata ha evitato duplicazioni tra le torri concrete e ha reso semplice l'aggiunta di nuove tipologie senza modificare la struttura esistente.

4.0.3 Michel Gad

Il mio contributo ha riguardato la gestione della mappa, la definizione dei percorsi e il supporto per le tre mappe multiple con difficoltà crescente, oltre alla realizzazione della schermata iniziale. Per la mappa ho progettato la struttura `MapData` come contenitore dei dati grezzi serializzati in JSON, e `MapLoader` come componente di caricamento che ne gestisce il parsing e la validazione, esponendo metodi sia per il filesystem che per il classpath. La costruzione del modello runtime `GameMapImpl` a partire da `MapData` ha permesso di tenere separati il formato di persistenza dalla rappresentazione operativa, rendendo semplice l'aggiunta di nuovi livelli senza modificare la logica. Per i per-

corsi ho definito i waypoint come liste di coordinate double, lasciando al nemico la responsabilità dell'avanzamento e mantenendo la mappa come sorgente di verità sulla topologia. Per la schermata iniziale ho curato la navigazione tra StartMenu, la schermata del regolamento e LevelSelect, delegando al MainController l'inizializzazione della partita al momento della selezione del livello. Sono soddisfatto di queste scelte: la separazione tra dati di mappa e modello runtime ha semplificato i test e reso il contenuto di gioco completamente configurabile senza toccare il codice.

4.0.4 Stefano Fedele

Il mio contributo ha riguardato il sistema di risorse, i punti vita della base, le condizioni di vittoria e sconfitta, il punteggio e il salvataggio della partita. Per le risorse ho definito l'interfaccia Player con i metodi addCoins, spendCoins e takeBaseDamage, centralizzando tutta la logica di validazione economica in un unico punto e rendendo impossibile spendere monete non disponibili senza gestione esplicita del caso. Per i punti vita della base ho collegato l'evento di raggiungimento del goal da parte di un nemico al metodo takeBaseDamage del Player, propagando la transizione a GameState.GAME OVER nel momento in cui isBaseAlive restituisce false. Per la vittoria ho implementato il controllo combinato su ondate completate e HP residui, delegando la transizione a LEVEL COMPLETE al GameModel al termine dell'ultima ondata. Per il punteggio ho definito l'interfaccia Score con logica di accumulo separata dal Player, mantenendo i due aspetti indipendenti e testabili in isolamento. Per il salvataggio ho realizzato SaveManager come componente statico che persiste su file il livello massimo sbloccato e il miglior punteggio, con metodi sovraccaricati per supportare path personalizzati nei test. Sono soddisfatto di queste scelte: la separazione tra economia, salute della base e punteggio ha reso ogni responsabilità ben delimitata e ha facilitato la scrittura di test precisi su ciascun comportamento.

4.1 Sviluppi futuri

4.1.1 Gameplay e contenuti

- **Nuove tipologie di torri:** Introduzione di torri con effetti speciali come veleno (*damage over time*), rallentamento permanente o danno ad area (*splash damage*), sfruttando la flessibilità dell'interfaccia **Tower** già predisposta per l'estensione.
- **Nuovi tipi di nemici:** Creazione di varianti dotate di resistenze specifiche (es. immunità al gelo, armatura contro i danni fisici) o comportamenti complessi (nemici volanti che ignorano parte del percorso, nemici che si dividono alla morte).
- **Più livelli e mappe:** Scalabilità del contenuto garantita dalla struttura **MapData/MapLoader** basata su JSON, che permette l'aggiunta di nuove mappe e percorsi multipli o ramificati senza dover modificare il codice di gioco.
- **Sistema di upgrade esteso:** Potenziamento delle torri oltre il livello 3, con lo sblocco progressivo di abilità passive dedicate (es. proiettili perforanti o incremento statico del raggio d'azione).

4.1.2 Meccaniche di gioco

- **Modalità endless:** Implementazione di ondate infinite a difficoltà crescente con classifica locale del punteggio, sfruttando la struttura del **SaveManager** già esistente per il salvataggio del *best score*.
- **Nuove abilità globali:** Ampliamento del repertorio di abilità (oltre alla pioggia di fuoco e al gelo globale) con l'introduzione di scudi temporanei per la base o bonus economici istantanei.

- **Sistema di achievement:** Definizione di obiettivi sbloccabili (es. completamento di un livello senza subire danni alla base o costruzione di un numero soglia di torri) per aumentare la rigiocabilità.
- **Economia avanzata:** Introduzione di meccaniche quali l'interesse percentuale sulle monete non spese tra un'ondata e la successiva, oppure un mercato dinamico con prezzi di acquisto e vendita variabili.

5 Guida Utente

Tutorial di TowerSiege

All'avvio dell'applicazione, ti troverai davanti al menu iniziale, che presenta quattro pulsanti principali:

- **Inizia Partita:** Avvia una nuova partita.
- **Regolamento:** Mostra le regole dettagliate del gioco.
- **Esci:** Esce dall'applicazione.



Regole del gioco: Premendo il pulsante **Rules**, verrà mostrata una schermata contenente una spiegazione dettagliata delle regole di gioco.

Uscita dall'applicazione: Premendo il pulsante **Esci**, ti verrà data la possibilità di chiudere l'applicazione.



Figura 2: Regolamento

5.1 Avvio di una nuova partita

Se premi sul pulsante **INIZIA PARTITA** ti troverai sulla seguente schermata

dove troverai la possibilità di scegliere tra tre livelli:

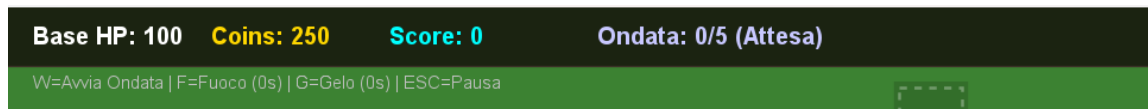


Figura 3: Livelli disponibili

5.2 Schermata di gioco

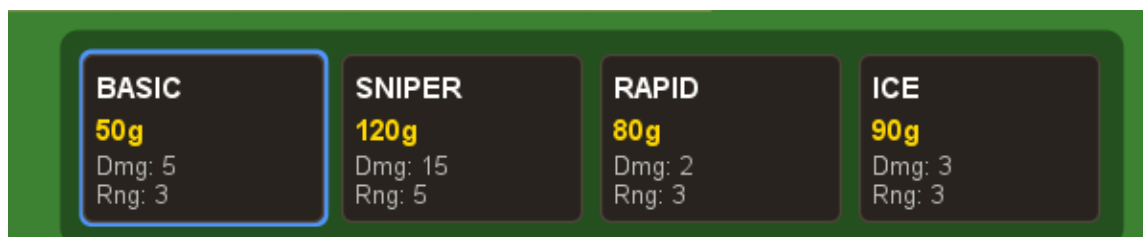
Ogni livello ha il proprio percorso ma ci sono elementi in comune:

Parte superiore della schermata di gioco:



Qui possiamo vedere gli HP della nostra base che diminuiscono ogni nemico che non riusciamo a uccidere, le MONETE che abbiamo inizialmente e che si accumulano uccidendo nemici, lo SCORE e il numero dell'ONDATA. Inoltre abbiamo dei comandi aggiunti come W per avviare un ondata e le due abilità F e G che rispettivamente fanno danno a tutti i nemici e li rallentano, mentre premendo ESC si mette in pausa il gioco

Parte inferiore della schermata di gioco:



Qui possiamo vedere il costo delle TORRI, i relativi danni e range.

5.3 Mappe di gioco

Ci sono tre diverse mappe/livelli di gioco, in ordine crescente di difficoltà.

Livello 1:



Livello 2:



Livello 3:



5.4 Tipologia di nemici

Ci sono tre tipologie di nemici con caratteristiche diverse.
BASIC, FAST, TANK.

NB: BASIC e TANK hanno la stessa immagine.





5.5 Tipologia di torri

Ci sono 4 tipologie di torri con diverse abilità, diverso costo, diverso attacco e diversa velocità di attacco.



Figura 4: Magic crystal



Figura 5: Mage hero



Figura 6: Knight hero

5.6 Pulsante di controllo

Durante il gioco possiamo decidere di mettere in pausa il livello per tornare al menu o per altri motivi; si può inoltre decidere di riavviare il livello se si è scontenti di come si sono gestite le torri e i soldi oppure se è stato preso troppo danno alla base. L'ultima opzione è invece quella di tornare al menu iniziale.



PAUSA

RIPRENDI

RIAVVIA LIVELLO

TORNA AL MENU